DinoSAT

FORMAL VERIFICATION REPORT

Glamtoon

2026-10-11

https://dinosat.com

Table of Contents

Verification Status Legend

Threat Model

- Assets at Risk
- Actors
- Trust Assumptions
- Attack Vectors Considered

Results Summary

Detailed Results

- PriceLemmasTest
- BitmapLemmasTest
- TraitSchemaLemmasTest
- PackingLemmasTest
- RoyaltyLemmasTest
- GlamToonPriceSymbolicTest
- TreasuryAccessSymbolicTest
- GlamToonOwnerLabelSymbolicTest
- GlamToonOwnerDecodingSymbolicTest
- GlamToonTransferSymbolicTest
- GlamToonWithdrawSymbolicTest
- TreasurySweepSymbolicTest
- TreasuryTokenSweepSymbolicTest
- GlamToonReentrancySymbolicTest
- GlamToonDeploymentSymbolicTest
- GlamToonGreenSymbolicTest
- GlamToonGiftSymbolicTest
- GlamToonBlueSaleSymbolicTest
- GlamToonBlueGuardsSymbolicTest
- SystemFlowSymbolicTest
- GlamToonViewGuardsSymbolicTest
- GlamToonRoyaltySymbolicTest
- TreasuryAnyCallSymbolicTest
- GlamToonHistorySymbolicTest
- GlamToonEtherFlowSymbolicTest
- TraitDataSymbolicTest

GlamToonMetadataSymbolicTest

GlamToonPackingSymbolicTest

GlamToonTraitReadersSymbolicTest

Properties Verified

P0 -- Critical

P1 -- High

P2 -- Medium

Cross-Contract

Coverage Gap Analysis

Methodology

Symbolic Execution

Proof Techniques Applied

Z3-Hostile Patterns Detected

Fuzz Testing (Complementary)

Vacuity Check

Seeded-Defect Check

Input Domain Bounds

Soundness Caveats

Proof Domain Boundaries

Abstract Specification Equivalence

Compositional Soundness

Halmos Verification Scope

Fuzz Limitations

Conclusion

Disclaimer

Protocol:	Glamtoon
Date:	2026-10-11
Tool:	Halmos v0.3.3 (Z3 SMT Solver)
Solidity:	0.8.37 (cancun)
Protocol type:	Token
Scope:	6 contracts, 509 nSLOC
Methodology:	Symbolic execution with exhaustive input space exploration

Verification Status Legend

Status	Meaning
PROVEN	Z3 exhausted all paths within the vm.assume input domain -- no counterexample exists. Mathematical proof.
PROVEN (composed)	End-to-end test is FUZZ, but all building-block lemmas are individually PROVEN. Compositional proof.
FUZZ	Property verified empirically via Forge fuzz testing (1,000,000 runs, 0 failures). Not a mathematical proof.
COUNTEREXAMPLE	Z3 found concrete input values that violate the assertion. Potential vulnerability -- see Detailed Results.
TIMEOUT	Z3 could not resolve within the solver timeout. Reclassified to FUZZ for empirical verification.

Threat Model

Assets at Risk

- Sale proceeds -- ether paid for blue editions, held by GlamToon until anyone calls withdraw, then by Treasury until anyone calls sweep. At most 250 ether over the whole sale.
- Buyer overpayment -- whatever a buyer sends above the tier price, which _mintBlue must return in the same transaction.
- Royalties -- ether and ERC-20 tokens marketplaces pay to Treasury, swept to the beneficiary by sweep and sweepToken.
- Token ownership -- every blue and green token, and the rule that only its owner or an approved account moves it.
- Supply integrity -- one blue per character and 10,000 in all; one green per address per character; generations issued in order; no two tokens sharing an id.
- Permanent configuration -- the payee, the sale start, the royalty and the three trait data contracts, fixed in the constructor with no setter to correct them.
- On-chain metadata -- tokenURI and the art addresses, which are built on chain and cannot be repaired after deployment.

Actors

- Buyer / minter -- any account; pays for a blue or takes a free green. Adversarial: may underpay, set a hostile ceiling, mint from a contract that re-enters, or refuse its own refund.
- Gift recipient -- the to of mintBlueFor; may be the zero address or a contract with a hostile receiver hook.
- Anyone -- calls withdraw, sweep and sweepToken, which check no caller. Adversarial: tries to divert what they move.
- Token holder -- transfers and approves; harmed by an unauthorised transfer or a burned token.
- Treasury owner -- the only privileged key. Chooses the beneficiary and may hand the treasury on in two steps. Trusted for where sweeps go; proved to have no power over the token.
- Beneficiary -- receives sweeps. Not trusted with anything else.
- External contracts -- the payee behind owner(), ERC-20 tokens passed to sweepToken, and receiver hooks. Treated as hostile where a proof names them.

Trust Assumptions

- msg.sender is never the zero address. Nobody holds its key. OpenZeppelin's ERC-721 skips its owner check for that caller and Ownable2Step would let it accept a handover nobody offered; one proof pins that this is the only caller the owner check lets through.
- The treasury owner sets the beneficiary in good faith. A malicious owner can redirect future sweeps; that is the key's stated purpose and is out of scope.
- Accounts used as callers and recipients have no code, except in the reentrancy proofs, which use the project's four attacker contracts. Arbitrary attacker code is out of scope for symbolic execution.
- The trait data in TraitData0, TraitData1 and TraitNames is the data the art pipeline produced. The proofs establish that the token pins and reads it correctly, not that the traits describe the art.
- The EVM, the Solidity 0.8.37 compiler and the OpenZeppelin v5 library behave as specified. Library code is executed symbolically along with the contracts, not assumed.
- Constructor validation: the payee is not zero and not the token itself, and the sale starts at most 90 days after deployment (GlamToon.sol:187-188).

Attack Vectors Considered

Attack Vector	Properties	Mitigated By
Mint a blue for less than its price	FB-3, PD-1	DIRECT guard and conservation proofs
Keep a buyer's overpayment	PD-1, FB-4	DIRECT conservation over every amount
Sell one character twice	SM-1, CC-1	DIRECT guard proof, bitmap lemmas
Mint past the 10,000 supply	TOK-3	DIRECT guard proof on the counter
Take two greens of one character	SM-1	DIRECT, incl. after giving the token away
Two tokens with one id / green on a blue id	NFT-1, NFT-3	DECOMPOSE division-free lemmas
Token id overflow wraps onto another id	AS-1, NFT-1	DIRECT in range, FUZZ past it
Divert a withdraw or a sweep	CC-3, AC-2	DIRECT conservation over every caller
Ether leaves the token some other way	PD-1, CC-3	ENUMERATE over every function
Stranger repoints or takes the treasury	AC-1, AC-4, AC-5	DIRECT and ENUMERATE over every function
Treasury left with no owner	AC-3	DIRECT, renounce always reverts
Unauthorised transfer or approval	AC-1, TOK-2, TOK-5	DIRECT over every caller
Burn, or undo a sale or a claim	AC-3, TOK-4	ENUMERATE over every function
Re-enter through the hook or the refund	NFT-4	DIRECT with attacker contracts
Hostile payee corrupts owner()	AS-4, CC-2	DIRECT over every returned word
Wrong trait data pinned at deployment	NFT-6, AC-3	DIRECT over every hash
Out-of-range read returns another record	SM-4, AS-3	DIRECT guards, ENUMERATE boundaries
Metadata reverts or names the wrong art	NFT-5	ENUMERATE boundary ids, FUZZ every id
Royalty above the sale price	FB-1, FB-2	DIRECT lemma and formula match

Results Summary

Contract	File	Tests	Proven	Fuzz	Status
PriceLemmasTest	PriceLemmas.t.sol	10	10	0	ALL PROVEN
BitmapLemmasTest	BitmapLemmas.t.sol	7	7	0	ALL PROVEN
TraitSchemaLemmasTest	TraitSchemaLemmas.t.sol	7	7	0	ALL PROVEN
PackingLemmasTest	PackingLemmas.t.sol	7	7	0	ALL PROVEN
RoyaltyLemmasTest	RoyaltyLemmas.t.sol	7	4	3	ALL VERIFIED
GlamToonPriceSymbolicTest	GlamToonPriceSymbolic.t.sol	5	5	0	ALL PROVEN
TreasuryAccessSymbolicTest	TreasuryAccessSymbolic.t.sol	9	9	0	ALL PROVEN
GlamToonOwnerLabelSymbolicTest	GlamToonOwnerLabelSymbolic.t.sol	5	5	0	ALL PROVEN
GlamToonOwnerDecodingSymbolicTest	GlamToonOwnerDecodingSymbolic.t.sol	6	6	0	ALL PROVEN
GlamToonTransferSymbolicTest	GlamToonTransferSymbolic.t.sol	11	11	0	ALL PROVEN
GlamToonWithdrawSymbolicTest	GlamToonWithdrawSymbolic.t.sol	5	5	0	ALL PROVEN
TreasurySweepSymbolicTest	TreasurySweepSymbolic.t.sol	5	5	0	ALL PROVEN
TreasuryTokenSweepSymbolicTest	TreasuryTokenSweepSymbolic.t.sol	6	6	0	ALL PROVEN
GlamToonReentrancySymbolicTest	GlamToonReentrancySymbolic.t.sol	5	5	0	ALL PROVEN
GlamToonDeploymentSymbolicTest	GlamToonDeploymentSymbolic.t.sol	9	9	0	ALL PROVEN
GlamToonGreenSymbolicTest	GlamToonGreenSymbolic.t.sol	10	10	0	ALL PROVEN
GlamToonGiftSymbolicTest	GlamToonGiftSymbolic.t.sol	5	5	0	ALL PROVEN
GlamToonBlueSaleSymbolicTest	GlamToonBlueSaleSymbolic.t.sol	7	7	0	ALL PROVEN
GlamToonBlueGuardsSymbolicTest	GlamToonBlueGuardsSymbolic.t.sol	8	8	0	ALL PROVEN
SystemFlowSymbolicTest	SystemFlowSymbolic.t.sol	5	5	0	ALL PROVEN
GlamToonViewGuardsSymbolicTest	GlamToonViewGuardsSymbolic.t.sol	10	10	0	ALL PROVEN
GlamToonRoyaltySymbolicTest	GlamToonRoyaltySymbolic.t.sol	6	6	0	ALL PROVEN
TreasuryAnyCallSymbolicTest	TreasuryAnyCallSymbolic.t.sol	8	8	0	ALL PROVEN
GlamToonHistorySymbolicTest	GlamToonHistorySymbolic.t.sol	11	11	0	ALL PROVEN
GlamToonEtherFlowSymbolicTest	GlamToonEtherFlowSymbolic.t.sol	11	11	0	ALL PROVEN
TraitDataSymbolicTest	TraitDataSymbolic.t.sol	10	8	2	ALL VERIFIED
GlamToonMetadataSymbolicTest	GlamToonMetadataSymbolic.t.sol	12	9	3	ALL VERIFIED
GlamToonPackingSymbolicTest	GlamToonPackingSymbolic.t.sol	6	6	0	ALL PROVEN
GlamToonTraitReadersSymbolicTest	GlamToonTraitReadersSymbolic.t.sol	9	9	0	ALL PROVEN
TOTAL		222	214	8	

Verdict: 214 formal proofs, 8 fuzz-verified, 0 counterexamples found.

All test files are in test/dinosat/. The 29 test contracts hold 222 test_check_ proofs and 119 test_fuzz_ wrappers.

Detailed Results

PriceLemmasTest

File: test/dinosat/PriceLemmas.t.sol

10 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_lemma_payment_splits_exactly(uint256,uint256) (paths: 4, time: 5.06s) -- PD-1/AS-3: For any payment that covers the price, the price and the change add up to what was sent, the change never underflows, and the buyer always pays something
- [PASS] test_check_lemma_price_monotonic(uint256,uint256) (paths: 4, time: 1.91s) -- FB-5: A later sale never costs less than an earlier one
- [PASS] test_check_lemma_price_steps_only_at_tier_boundary(uint256) (paths: 7, time: 2.02s) -- SM-5: One more sale raises the price by exactly one step when it completes a tier of 2,500, and leaves it alone otherwise
- [PASS] test_check_lemma_price_within_bounds(uint256) (paths: 3, time: 0.85s) -- FB-3/FB-4: No blue costs less than 0.01 ether or more than 0.04
- [PASS] test_check_lemma_sellout_revenue_is_250_ether() (paths: 1, time: 0.00s) -- PD-1: Four tiers of 2,500 at the four prices come to exactly 250 ether, and the four tiers are the whole supply
- [PASS] test_check_lemma_tier_index_bounded(uint256) (paths: 2, time: 0.32s) -- AS-1: Below the sellout the tier index is 0 to 3, so the multiplier is at most 4 and the product cannot overflow
- [PASS] test_check_theorem_price_tier0(uint256) (paths: 3, time: 0.85s) -- FB-3/FB-4: The first 2,500 blues cost 0.01 ether
- [PASS] test_check_theorem_price_tier1(uint256) (paths: 3, time: 0.85s) -- FB-3/FB-4: Sales 2,500 to 4,999 cost 0.02 ether
- [PASS] test_check_theorem_price_tier2(uint256) (paths: 3, time: 0.85s) -- FB-3/FB-4: Sales 5,000 to 7,499 cost 0.03 ether
- [PASS] test_check_theorem_price_tier3(uint256) (paths: 3, time: 0.85s) -- FB-3/FB-4: Sales 7,500 to 9,999 cost 0.04 ether

Composed proofs:

- The price schedule -- PROVEN via composition of four branch theorems:
  - test_check_theorem_price_tier0 [PASS] + tier1 [PASS] + tier2 [PASS] + tier3 [PASS], with test_check_lemma_tier_index_bounded [PASS]
  - Composition: the four ranges [0, 2499], [2500, 4999], [5000, 7499], [7500, 9999] are disjoint and their union is every value priceAt accepts.

BitmapLemmasTest

File: test/dinosat/BitmapLemmas.t.sol

7 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_lemma_bit_is_single_and_nonzero(uint256) (paths: 3, time: 0.12s) -- CC-1: The mask a sale sets is exactly one bit, never none
- [PASS] test_check_lemma_bitmap_capacity_is_tight() (paths: 1, time: 0.00s) -- SM-4: Forty words hold all ten thousand characters, and thirty nine would not
- [PASS] test_check_lemma_positions_are_injective(uint256,uint256) (paths: 4, time: 0.12s) -- CC-1: Two characters share a word

and a bit exactly when they are the same character, so no sale can mark another's bit

- [PASS] test_check_lemma_setting_a_bit_keeps_the_rest(uint256,uint256) (paths: 4, time: 0.13s) -- AC-3: Setting a character's bit sets that bit, leaves every other bit of the word as it was, and clears nothing

- [PASS] test_check_lemma_shift_and_mask_are_div_and_mod(uint256) (paths: 1, time: 0.00s) -- CC-1: The shift and the mask are division and remainder by 256, which is how a reader outside the contract finds a character

- [PASS] test_check_lemma_word_index_in_bitmap(uint256) (paths: 1, time: 0.00s) -- SM-4: Every character's word is one of the forty the bitmap has

- [PASS] test_check_lemma_word_range_guard_is_exact(uint256,uint256) (paths: 7, time: 0.13s) -- SM-4/AS-3: The check refuses exactly the ranges that leave the bitmap

TraitSchemaLemmasTest

File: test/dinosat/TraitSchemaLemmas.t.sol

7 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_lemma_chunk_index_is_zero_or_one(uint256) (paths: 3, time: 0.43s) -- SM-4: The chunk a character is read from is 0 or 1, and it is 0 exactly for the first five thousand

- [PASS] test_check_lemma_fields_rebuild_the_record(uint256) (paths: 1, time: 0.00s) -- NFT-6: The four values put back at their shifts are the record, so no bit is read twice and none is ignored

- [PASS] test_check_lemma_layout_constants_agree() (paths: 1, time: 0.00s) -- NFT-6: The layout constants agree with each other

- [PASS] test_check_lemma_name_slots_stay_in_table(uint40) (paths: 2, time: 0.00s) -- SM-4: Whatever a record holds, every name slot _attributes asks for exists, so no record can make the metadata revert

- [PASS] test_check_lemma_record_offset_in_bounds_chunk0(uint256) (paths: 1, time: 0.00s) -- AS-3: For every character the chunk accepts, both bytes of its record lie inside the chunk and the subtraction that finds them cannot underflow

- [PASS] test_check_lemma_record_offset_in_bounds_chunk1(uint256) (paths: 1, time: 0.00s) -- AS-3: The same for characters 5000 to 9999

- [PASS] test_check_lemma_trait_values_fit_their_type(uint40) (paths: 2, time: 0.00s) -- AS-4: Whatever a record holds, each of its four values is at most fifteen, so the cast to uint8 in traitsOf cannot truncate

PackingLemmasTest

File: test/dinosat/PackingLemmas.t.sol

7 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_lemma_blue_iff_generation_zero(uint256,uint256) (paths: 2, time: 0.43s) -- NFT-3: An id is below 10,000 exactly when its generation is zero, so no green generation can land on a blue id

- [PASS] test_check_lemma_decode_image_in_range(uint256) (paths: 2, time: 0.84s) -- NFT-2: Decoding any token id at all names a character inside the collection

- [PASS] test_check_lemma_decode_recomposes(uint256) (paths: 3, time: 13.68s) -- NFT-2: The two halves of a decoded id put together are the id

- [PASS] test_check_lemma_id_rises_with_generation(uint256,uint256,uint256,uint256) (paths: 4, time: 1.90s) -- NFT-1: Lemma: a higher generation always has a higher token id, whichever two characters are compared

- [PASS] test_check_lemma_round_trip(uint256,uint256) (paths: 3, time: 8.79s) -- NFT-2: Decoding an encoded pair gives the pair back
- [PASS] test_check_lemma_same_generation_keeps_characters_apart(uint256,uint256,uint256) (paths: 1, time: 0.00s) -- NFT-1: Lemma: inside one generation two ids are equal exactly when the characters are
- [PASS] test_check_theorem_no_collision(uint256,uint256,uint256,uint256) (paths: 4, time: 8.15s) -- NFT-1: Theorem: two pairs with the same token id are the same pair, end to end

Composed proofs:

- test_check_theorem_no_collision -- [PASS] end to end for generations below 2^16, and PROVEN via composition for every generation below 2^200:
  - test_check_lemma_id_rises_with_generation [PASS] + test_check_lemma_same_generation_keeps_characters_apart [PASS]
  - Composition: either one generation is higher, and the first lemma orders the two ids (applied once each way), or the generations are equal, and the second lemma says the ids match only for the same character. The three cases cover every pair. The lemmas use wrapping arithmetic, so they also show no id in that range wraps.

RoyaltyLemmasTest

File: test/dinosat/RoyaltyLemmas.t.sol

4 formally proven, 3 fuzz-verified

Proven:

- [PASS] test_check_lemma_royalty_never_exceeds_price(uint256) (paths: 3, time: 2.61s) -- FB-1: The royalty is never more than the sale price
- [PASS] test_check_lemma_royalty_product_fits(uint256) (paths: 3, time: 0.12s) -- AS-1: The price times the rate cannot wrap
- [PASS] test_check_lemma_royalty_rate_is_below_one() (paths: 1, time: 0.00s) -- FB-2: The rate is two percent, below the whole
- [PASS] test_check_lemma_royalty_rounds_to_zero_below_fifty_wei(uint256) (paths: 3, time: 2.71s) -- PD-2: Below fifty wei the royalty rounds down to nothing, so rounding only ever favours the payer

Fuzz-verified:

- [FUZZ] test_check_lemma_royalty_floor_identity(uint256) (1,000,000 runs, 0 failures) -- PD-3: The royalty is the floor of two percent. Halmos: TIMEOUT at 30 s and again at 60 s.
- [FUZZ] test_check_lemma_royalty_is_one_fiftieth(uint256) (1,000,000 runs, 0 failures) -- PD-3: Two percent is one fiftieth, the same amount by a smaller divisor. Halmos: TIMEOUT at 30 s and again at 60 s.
- [FUZZ] test_check_lemma_royalty_monotonic(uint256,uint256) (1,000,000 runs, 0 failures) -- FB-5: A higher sale price never pays a lower royalty. Halmos: TIMEOUT at 30 s and again at 60 s.

GlamToonPriceSymbolicTest

File: test/dinosat/GlamToonPriceSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_a_sale_moves_the_price_along_the_schedule(uint256,uint256) (paths: 47, time: 14.03s) -- SM-5: A sale moves the quoted price to the next point of the schedule, which is the same price or one step more, never less
- [PASS] test_check_currentPrice_follows_the_sales_counter(uint256) (paths: 10, time: 1.21s) -- SM-5: currentPrice is the schedule at blueMinted, and refuses with SoldOut once that reaches the supply

- [PASS] test_check_priceAt_matches_schedule(uint256) (paths: 9, time: 1.18s) -- FB-3: priceAt answers the published schedule for every number of sales short of the sellout, and never reverts there
- [PASS] test_check_priceAt_refuses_once_sold_out(uint256) (paths: 1, time: 0.00s) -- TOK-3: From the ten thousandth sale on, priceAt refuses with SoldOut, for every count a word can hold
- [PASS] test_check_price_constants_match_the_lemmas() (paths: 1, time: 0.01s) -- FB-3: The constants the pure lemma contracts restate are the ones the deployed contract reports

TreasuryAccessSymbolicTest

File: test/dinosat/TreasuryAccessSymbolic.t.sol

9 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_a_new_offer_replaces_the_old_one(address,address) (paths: 3, time: 0.01s) -- AC-4: A second offer replaces the first
- [PASS] test_check_accepting_moves_the_key_and_clears_the_offer(address,address) (paths: 3, time: 0.02s) -- AC-4/AC-5: Accepting moves the ownership, clears the offer, takes the old owner's power away and gives it to the new one
- [PASS] test_check_an_offer_does_not_hand_over(address,address) (paths: 3, time: 0.01s) -- AC-5: An offer names a pending owner and changes nothing else
- [PASS] test_check_constructor_refuses_zero_addresses(address,address) (paths: 5, time: 0.01s) -- AC-3: A treasury is built exactly when neither its owner nor its beneficiary is the zero address, and then reports both as given, with no offer pending
- [PASS] test_check_only_the_offered_account_can_accept(address,address) (paths: 3, time: 0.01s) -- AC-5: Only the account that was offered the treasury can accept it
- [PASS] test_check_owner_moves_the_beneficiary_but_not_to_nobody(address) (paths: 3, time: 0.01s) -- AC-1: The owner moves the beneficiary to any address but the zero one, which is refused with ZeroAddress
- [PASS] test_check_renouncing_is_always_refused(address) (paths: 2, time: 0.00s) -- AC-3: Renouncing is refused with CannotRenounce for every caller, the owner included
- [PASS] test_check_stranger_cannot_move_the_beneficiary(address,address) (paths: 3, time: 0.01s) -- AC-1: Nobody but the owner can move the beneficiary, to anywhere
- [PASS] test_check_stranger_cannot_offer_ownership(address,address) (paths: 3, time: 0.01s) -- AC-1: Nobody but the owner can offer the treasury to anyone

GlamToonOwnerLabelSymbolicTest

File: test/dinosat/GlamToonOwnerLabelSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_label_follows_a_two_step_handover(address) (paths: 2, time: 0.02s) -- CC-2: The label stays put while a handover is only offered, and moves when the new owner accepts, whoever that is
- [PASS] test_check_label_holder_cannot_mint_below_the_price(uint256,uint256,uint256) (paths: 29, time: 2.83s) -- AC-2: The address the label names pays the same price as everybody else
- [PASS] test_check_label_holder_cannot_move_a_holders_token(uint256,address) (paths: 3, time: 0.09s) -- AC-2: The address the label names cannot move, or approve itself for, a token it does not hold
- [PASS] test_check_label_holder_cannot_redirect_a_withdraw(uint256) (paths: 1, time: 0.01s) -- AC-2: The address the label names cannot steer a withdraw

- [PASS] test_check_label_is_the_treasury_owner() (paths: 1, time: 0.00s) -- CC-2: The token reports the treasury's owner as its own

GlamToonOwnerDecodingSymbolicTest

File: test/dinosat/GlamToonOwnerDecodingSymbolic.t.sol

6 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_label_is_nobody_when_the_answer_is_empty() (paths: 1, time: 0.02s) -- AS-4: A payee that answers with nothing, with or without code, reads as nobody
- [PASS] test_check_label_is_nobody_when_the_answer_is_long() (paths: 1, time: 0.01s) -- AS-4: A payee that answers two words reads as nobody, even though the first of them is a clean address
- [PASS] test_check_label_is_nobody_when_the_answer_is_short() (paths: 1, time: 0.01s) -- AS-4: A payee that answers one byte short of a word reads as nobody
- [PASS] test_check_label_is_nobody_when_the_payee_reverts() (paths: 1, time: 0.01s) -- AS-4: A payee that reverts reads as nobody
- [PASS] test_check_label_reads_a_clean_address(uint256) (paths: 1, time: 0.02s) -- AS-4: A word that is an address reads as that address, for every one of them, and owner() does not revert
- [PASS] test_check_label_refuses_a_dirty_word(uint256) (paths: 1, time: 0.00s) -- AS-4: A word with anything above its low 160 bits is not an address, and reads as nobody rather than being truncated into one

GlamToonTransferSymbolicTest

File: test/dinosat/GlamToonTransferSymbolic.t.sol

11 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_approval_alone_moves_nothing(address) (paths: 2, time: 0.01s) -- TOK-5: Approving a spender records the approval and moves nothing
- [PASS] test_check_approved_spender_moves_it_once(address,address) (paths: 3, time: 0.07s) -- TOK-5: An approved spender can move the token once
- [PASS] test_check_only_the_zero_address_escapes_the_owner_check(address,address) (paths: 4, time: 0.04s) -- AC-1: The one caller the owner check lets through is the zero address
- [PASS] test_check_operator_approval_is_scoped_to_its_grantor(address,address) (paths: 4, time: 0.08s) -- AC-1: An operator approval covers the tokens of the account that granted it and nobody else's
- [PASS] test_check_owner_moves_its_token(address) (paths: 3, time: 0.05s) -- TOK-2: The owner can move its token to any address but the zero one
- [PASS] test_check_stranger_cannot_approve(address,address) (paths: 3, time: 0.02s) -- AC-1: Nobody but its owner can approve a spender for a token
- [PASS] test_check_stranger_cannot_move_a_token(address,address) (paths: 4, time: 0.09s) -- AC-1: Nobody but its owner can move a token, to anywhere, by either transfer function
- [PASS] test_check_transfer_changes_no_supply(address) (paths: 2, time: 0.03s) -- TOK-2: A transfer leaves the supply, the counters, the sold bit and the green claim as they were
- [PASS] test_check_transfer_to_nobody_is_refused() (paths: 1, time: 0.00s) -- TOK-4: A token cannot be sent to the zero address by its owner, by either transfer function

- [PASS] test_check_unminted_token_cannot_move(address,address,address,uint256) (paths: 5, time: 0.03s) -- AC-1: A token that was never minted cannot be moved by anybody
- [PASS] test_check_wrong_from_is_refused(address,address) (paths: 4, time: 0.03s) -- AC-1: Naming the wrong owner is refused even by the real one

GlamToonWithdrawSymbolicTest

File: test/dinosat/GlamToonWithdrawSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_withdraw_leaves_the_collection_alone(address,uint256) (paths: 3, time: 0.18s) -- AC-2: A withdraw leaves every token, counter and claim as it was
- [PASS] test_check_withdraw_moves_exactly_the_takings(uint256,uint256,uint256) (paths: 38, time: 3.12s) -- CC-3: A sale followed by a withdraw moves exactly the price to the treasury
- [PASS] test_check_withdraw_pays_only_the_treasury(address,uint256) (paths: 2, time: 0.02s) -- CC-3: Whoever calls it, a withdraw empties the contract into the treasury, for every balance, and the caller's own balance does not move
- [PASS] test_check_withdraw_refuses_ether(address,uint256,uint256) (paths: 2, time: 0.02s) -- PD-1: withdraw takes no ether
- [PASS] test_check_withdraw_refuses_when_empty(address) (paths: 2, time: 0.00s) -- PD-1: An empty contract refuses a withdraw with NothingToWithdraw, whoever asks

TreasurySweepSymbolicTest

File: test/dinosat/TreasurySweepSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_sweep_follows_a_moved_beneficiary(address,uint256) (paths: 2, time: 0.03s) -- CC-3: After the owner repoints the treasury, the next sweep pays the new beneficiary all of it and the old one nothing
- [PASS] test_check_sweep_pays_only_the_beneficiary(address,uint256) (paths: 2, time: 0.02s) -- CC-3: Whoever calls it, a sweep empties the treasury into the beneficiary, for every balance, and the caller's own balance does not move
- [PASS] test_check_sweep_refuses_ether(address,uint256,uint256) (paths: 2, time: 0.02s) -- PD-1: sweep takes no ether
- [PASS] test_check_sweep_refuses_when_empty(address) (paths: 2, time: 0.00s) -- PD-1: An empty treasury refuses a sweep with NothingToSweep, whoever asks
- [PASS] test_check_treasury_accepts_ether_from_anyone(address,uint256,uint256) (paths: 3, time: 0.03s) -- PD-1: The treasury accepts ether from anybody, in any amount, keeps exactly what was sent, and changes nothing else by receiving it

TreasuryTokenSweepSymbolicTest

File: test/dinosat/TreasuryTokenSweepSymbolic.t.sol

6 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_the_two_sweeps_do_not_touch_each_other(uint256,uint256) (paths: 2, time: 0.04s) -- PD-1: Sweeping a

token leaves the treasury's ether where it was, and sweeping ether leaves its tokens where they were

- [PASS] test_check_token_sweep_follows_a_moved_beneficiary(address,uint256) (paths: 2, time: 0.03s) -- CC-3: After the owner repoints the treasury, a token sweep pays the new beneficiary all of it and the old one nothing

- [PASS] test_check_token_sweep_handles_a_token_that_returns_nothing(uint256) (paths: 1, time: 0.02s) -- CC-3: A token whose transfer returns nothing, the way USDT's does, is swept like any other

- [PASS] test_check_token_sweep_pays_only_the_beneficiary(address,uint256) (paths: 2, time: 0.02s) -- CC-3: Whoever calls it, a token sweep moves the treasury's whole balance to the beneficiary and none to the caller

- [PASS] test_check_token_sweep_refuses_a_token_that_answers_false(uint256) (paths: 1, time: 0.01s) -- CC-3: A token whose transfer answers false without moving anything makes the sweep revert, rather than report tokens as sent that are still in the treasury

- [PASS] test_check_token_sweep_refuses_when_empty(address) (paths: 2, time: 0.00s) -- PD-1: A treasury that holds none of a token refuses to sweep it with NothingToSweep, whoever asks

GlamToonReentrancySymbolicTest

File: test/dinosat/GlamToonReentrancySymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_green_reentry_from_a_blue_sale_is_refused(uint256,uint256) (paths: 2, time: 0.05s) -- NFT-4: The same re-entry from inside a blue sale is refused, the sale reverts, and the payment comes back in full

- [PASS] test_check_green_reentry_from_the_hook_is_refused(uint256,uint256) (paths: 1, time: 0.03s) -- NFT-4: A receiver that asks for a second green from inside its hook is refused by the lock, the whole mint reverts, and no generation or claim is spent on either character

- [PASS] test_check_minter_without_a_hook_spends_nothing(uint256) (paths: 1, time: 0.02s) -- NFT-4/SM-1: A contract with no receiver hook cannot take a green, and trying does not spend its claim or a generation

- [PASS] test_check_unreturnable_change_refuses_the_sale(uint256,uint256) (paths: 3, time: 0.04s) -- PD-1: A buyer whose contract refuses ether completes a sale exactly when it sent the exact price

- [PASS] test_check_withdraw_reentry_from_the_refund_is_refused(uint256,uint256) (paths: 3, time: 0.12s) -- NFT-4: A buyer that calls withdraw from inside its refund is refused by the lock

GlamToonDeploymentSymbolicTest

File: test/dinosat/GlamToonDeploymentSymbolic.t.sol

9 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_constructor_forwards_its_seed_to_the_payee(uint256) (paths: 2, time: 0.12s) -- PD-1/CC-3: What is sent with the deployment goes straight to the payee and none of it stays in the token, for every amount

- [PASS] test_check_constructor_pins_what_it_was_given(uint64) (paths: 2, time: 0.14s) -- AC-3: A token reports exactly what it was built with: the payee, the start, the three data contracts, a two percent royalty to the payee, and an empty sale

- [PASS] test_check_constructor_refuses_a_chunk_it_cannot_read() (paths: 1, time: 0.12s) -- NFT-6: A chunk that reports the right hash but cannot answer for a record is refused

- [PASS] test_check_constructor_refuses_a_wrong_chunk0_hash(bytes32) (paths: 1, time: 0.06s) -- NFT-6: A first chunk that reports any hash but the expected one is refused with BadTraitData

- [PASS] test_check_constructor_refuses_a_wrong_chunk1_hash(bytes32) (paths: 1, time: 0.05s) -- NFT-6: The same for the

second chunk

- [PASS] test_check_constructor_refuses_a_wrong_names_hash(bytes32) (paths: 1, time: 0.10s) -- NFT-6: A name table that reports any hash but the expected one is refused with BadTraitNames

- [PASS] test_check_constructor_refuses_a_zero_payee(uint64) (paths: 2, time: 0.00s) -- AC-3: A token cannot be built with nobody as its payee, whatever the start time

- [PASS] test_check_constructor_refuses_swapped_chunks() (paths: 1, time: 0.05s) -- NFT-6: The two real chunks in the wrong order are refused, so the first five thousand characters cannot be given the traits of the last

- [PASS] test_check_constructor_start_window(uint64,uint64) (paths: 4, time: 0.12s) -- SM-1: A token is built exactly when its start is at most ninety days after the deployment, for every start and every clock

GlamToonGreenSymbolicTest

File: test/dinosat/GlamToonGreenSymbolic.t.sol

10 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_claim_survives_giving_the_token_away(address,address,uint256) (paths: 4, time: 0.18s) -- SM-1: The claim records the mint and not the holding, so giving the token away does not buy a second one

- [PASS] test_check_claims_are_per_character(address,uint256,uint256) (paths: 2, time: 0.07s) -- SM-1: The claim is per character

- [PASS] test_check_first_green_is_generation_one(address,uint256) (paths: 2, time: 0.04s) -- SM-5: The first account to take a character gets generation one, whose id is the character's number plus ten thousand

- [PASS] test_check_generations_are_issued_in_order(address,address,uint256) (paths: 3, time: 0.07s) -- SM-5: Two accounts that want the same character are not in each other's way

- [PASS] test_check_green_is_free_and_refuses_ether(address,uint256,uint256) (paths: 2, time: 0.08s) -- PD-1: Green costs only gas

- [PASS] test_check_green_never_lands_on_a_blue_id(address,uint256) (paths: 2, time: 0.06s) -- NFT-3: A green mint lands outside the blue range, leaves the character's blue unsold, takes no ether, and the blue can still be bought afterwards

- [PASS] test_check_green_refuses_an_unknown_character(address,uint256) (paths: 3, time: 0.15s) -- SM-4: Asking for a character that does not exist is refused with InvalidImageId and changes nothing

- [PASS] test_check_green_refuses_before_the_sale_opens(uint256,address,uint256) (paths: 2, time: 0.03s) -- SM-1: Before SALE_START no green mints, for any account or character, and no claim is spent by trying

- [PASS] test_check_green_stays_open_after_blue_sells_out(address,uint256,uint256) (paths: 3, time: 0.15s) -- NFT-3: Green stays open after the blue edition sells out

- [PASS] test_check_one_green_per_address_per_character(address,uint256) (paths: 2, time: 0.05s) -- SM-1: A second attempt by the same account is refused with GreenAlreadyTaken and leaves the counters where they were, so a refused mint cannot burn a generation

GlamToonGiftSymbolicTest

File: test/dinosat/GlamToonGiftSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_gift_cannot_take_a_sold_character(address,uint256,uint256) (paths: 3, time: 0.15s) -- SM-1: A gift cannot take a character that has sold, whoever it is for and whatever is paid

- [PASS] test_check_gift_reaches_any_account(address,uint256) (paths: 2, time: 0.05s) -- AC-2: A gift reaches any account it is addressed to, including the payer's own
- [PASS] test_check_gift_sends_token_and_change_apart(uint256,uint256,uint256,uint256) (paths: 41, time: 20.04s) -- PD-1: The payer is charged the price and refunded the rest, the recipient receives the token and not a wei, for every amount, ceiling, character and point of the sale
- [PASS] test_check_gift_settles_exactly_when_every_guard_passes(uint256,uint256,uint256,uint256,uint256) (paths: 91, time: 9.14s) -- SM-1/TOK-3/FB-3/FB-4: A gift to an account settles exactly when every guard passes, the same condition as a purchase
- [PASS] test_check_gift_to_nobody_is_refused(uint256,uint256,uint256) (paths: 4, time: 0.13s) -- TOK-4: A gift addressed to nobody is refused, the payer keeps its ether, and the character stays unsold and buyable

GlamToonBlueSaleSymbolicTest

File: test/dinosat/GlamToonBlueSaleSymbolic.t.sol

7 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_any_account_can_buy(address,uint256) (paths: 2, time: 0.05s) -- AC-2: There is no allowlist
- [PASS] test_check_buyer_pays_exactly_the_price(uint256,uint256,uint256,uint256) (paths: 40, time: 10.40s) -- PD-1: A buyer is charged the price of the tier the sale is in and gets back everything above it, for every amount, every ceiling that allows the price, every character and every point of the sale
- [PASS] test_check_exact_payment_leaves_no_change(uint256,uint256) (paths: 23, time: 2.90s) -- PD-1: Paying the exact price leaves the buyer with nothing sent back and nothing owed
- [PASS] test_check_sale_leaves_green_alone(uint256) (paths: 2, time: 0.02s) -- CC-1: A blue sale leaves the green edition exactly as it was
- [PASS] test_check_sale_records_owner_supply_and_bit(uint256,uint256) (paths: 23, time: 2.02s) -- TOK-1/CC-1: A sale gives the buyer the token whose id is the character's number, adds one to the supply, and marks the character sold everywhere the contract reports it
- [PASS] test_check_sale_sets_only_its_own_bit(uint256,uint256) (paths: 1, time: 0.02s) -- CC-1: A sale sets its own character's bit and no other
- [PASS] test_check_two_characters_sell_independently(uint256,uint256) (paths: 2, time: 0.15s) -- SM-1: Two different characters sell to two buyers, in either order of ids, and each ends up with its own

GlamToonBlueGuardsSymbolicTest

File: test/dinosat/GlamToonBlueGuardsSymbolic.t.sol

8 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_refusal_reason_follows_the_guard_order(uint256,uint256,uint256,uint256,uint256,bool) (paths: 82, time: 9.73s) -- SM-1/TOK-3/FB-3/FB-4: When more than one guard would refuse a sale, the reason given is the first in a fixed order: not started, unknown character, sold out, already taken, price above the ceiling, underpaid
- [PASS] test_check_refuses_a_price_above_the_ceiling(uint256,uint256,uint256,uint256) (paths: 32, time: 3.29s) -- FB-4: A ceiling below the price refuses the sale with PriceTooHigh, however much was sent
- [PASS] test_check_refuses_a_second_sale(address,uint256,uint256,uint256) (paths: 2, time: 0.10s) -- SM-1: A character that has sold cannot be sold again, to any account, at any amount or ceiling, and stays with its buyer
- [PASS] test_check_refuses_an_unknown_character(uint256,uint256,uint256) (paths: 1, time: 0.01s) -- SM-4: A character number

of 10,000 or more is refused with InvalidImageId, whatever is paid

- [PASS] test_check_refuses_before_the_sale_opens(uint256,uint256,uint256,uint256) (paths: 1, time: 0.01s) -- SM-1: Before SALE_START a sale is refused with NotStarted, for every character, amount and ceiling, and the buyer keeps its ether

- [PASS] test_check_refuses_underpayment(uint256,uint256,uint256,uint256) (paths: 31, time: 3.38s) -- FB-3: Sending less than the price refuses the sale with InsufficientPayment and returns everything

- [PASS] test_check_refuses_when_sold_out(uint256,uint256,uint256,uint256) (paths: 1, time: 0.01s) -- TOK-3: Once ten thousand blues have sold, no character sells at any amount or ceiling

- [PASS] test_check_settles_exactly_when_every_guard_passes(uint256,uint256,uint256,uint256,uint256) (paths: 91, time: 10.55s) -- SM-1/TOK-3/FB-3/FB-4: A sale of an unsold character to an account settles exactly when every guard passes: the sale is open, the character exists, the supply is not exhausted, the ceiling allows the price and the price was sent

SystemFlowSymbolicTest

File: test/dinosat/SystemFlowSymbolic.t.sol

5 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_a_deployment_seed_reaches_the_beneficiary(uint256) (paths: 1, time: 0.24s) -- CC-3: What a deployment sends along reaches the beneficiary through the treasury, the two hops the deploy script uses to prove the treasury can be paid before anybody buys

- [PASS] test_check_a_moved_beneficiary_receives_the_next_takings(address,uint256) (paths: 2, time: 0.07s) -- CC-3: Once the owner repoints the treasury, the next sale's price reaches the new beneficiary and the old one gets nothing

- [PASS] test_check_sale_proceeds_reach_the_beneficiary(uint256,uint256,uint256,address,address) (paths: 39, time: 3.10s) -- CC-3: For every character, point of the sale and amount sent, the price ends up with the beneficiary and the change with the buyer, whoever calls withdraw and whoever calls sweep, and neither of them gains

- [PASS] test_check_system_ether_is_conserved(uint256,uint256,uint256) (paths: 28, time: 9.64s) -- CC-3: At every step of that path the buyer, the token, the treasury and the beneficiary hold between them exactly what the buyer started with

- [PASS] test_check_treasury_owner_gains_nothing_by_pushing(uint256,uint256) (paths: 25, time: 3.03s) -- AC-2: The treasury's owner pushing the money along itself gains nothing by it

GlamToonViewGuardsSymbolicTest

File: test/dinosat/GlamToonViewGuardsSymbolic.t.sol

10 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_blueTakenBits_answers_inside_0words(uint256) (paths: 1, time: 0.01s) -- SM-4: An empty range is answered with an empty array from every start up to and including the end of the bitmap

- [PASS] test_check_blueTakenBits_answers_inside_1word(uint256) (paths: 2, time: 0.01s) -- SM-4: Every single word inside the bitmap is answered, and it holds the sold character's bit exactly when it is that character's word

- [PASS] test_check_blueTakenBits_answers_inside_40words() (paths: 1, time: 0.03s) -- SM-4: The whole bitmap in one call holds the sold character's bit and no other

- [PASS] test_check_blueTakenBits_refuses_a_range_outside(uint256,uint256) (paths: 2, time: 0.01s) -- SM-4: blueTakenBits refuses every range that leaves the forty words with InvalidWordRange

- [PASS] test_check_blueTaken_answers_inside_and_refuses_outside(uint256) (paths: 2, time: 0.01s) -- SM-4: blueTaken refuses a character past the collection, and inside it answers true for the one that sold and false for every other

- [PASS] test_check_green_batch_readers_answer_empty_outside(uint256,address) (paths: 2, time: 0.01s) -- SM-4: The green batch readers do not refuse
- [PASS] test_check_tokenIdFor_refuses_an_unknown_character(uint256,uint256) (paths: 1, time: 0.00s) -- SM-4: tokenIdFor refuses a character past the collection with InvalidImageId, for every generation
- [PASS] test_check_traitChunk_refuses_an_unknown_chunk(uint256) (paths: 1, time: 0.00s) -- SM-4: traitChunk refuses any index but 0 and 1 with InvalidChunk
- [PASS] test_check_trait_readers_refuse_an_unknown_character(uint256) (paths: 1, time: 0.01s) -- SM-4: The trait readers refuse a character past the collection with InvalidImageId, one at a time or in a batch
- [PASS] test_check_valueNames_refuses_an_unknown_category(uint256) (paths: 1, time: 0.00s) -- SM-4: valueNames refuses any category but 0 to 3 with InvalidCategory

GlamToonRoyaltySymbolicTest

File: test/dinosat/GlamToonRoyaltySymbolic.t.sol

6 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_royalty_always_answers_in_range(uint256,uint256) (paths: 3, time: 36.43s) -- AS-1: royaltyInfo answers, rather than reverts, for every price up to a billion ether and every token id, and the royalty is never more than the price
- [PASS] test_check_royalty_is_the_same_for_every_token(uint256,uint256,uint256) (paths: 2, time: 0.01s) -- PD-3: The royalty does not depend on the token
- [PASS] test_check_royalty_is_two_percent_to_the_treasury(uint256,uint256) (paths: 2, time: 0.01s) -- PD-3/CC-3: The royalty on any sale is the price times two hundred over ten thousand, and its receiver is the treasury, for every token id
- [PASS] test_check_royalty_is_untouched_by_the_sale(uint256,uint256) (paths: 1, time: 0.04s) -- AC-3: Minting, selling and withdrawing leave the royalty where the constructor set it
- [PASS] test_check_royalty_refuses_a_price_that_would_overflow(uint256,uint256) (paths: 2, time: 13.52s) -- AS-1: A price so large that two hundred times it would not fit a word is refused
- [PASS] test_check_supported_interfaces_are_exactly_four(bytes4) (paths: 6, time: 0.01s) -- NFT-5: supportsInterface is true for ERC-165, ERC-721, ERC-721 metadata and ERC-2981, and false for every other four bytes

TreasuryAnyCallSymbolicTest

File: test/dinosat/TreasuryAnyCallSymbolic.t.sol

8 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_stranger_changes_nothing_acceptOwnership(address,uint256) (paths: 3, time: 0.07s) -- AC-1/PD-1: For acceptOwnership, with no offer made
- [PASS] test_check_stranger_changes_nothing_anySelector(address,uint256,bytes4) (paths: 21, time: 0.35s) -- AC-1/PD-1: For any four byte selector sent alone: every function that takes no argument, and every selector the treasury does not have
- [PASS] test_check_stranger_changes_nothing_plainEther(address,uint256) (paths: 2, time: 0.06s) -- AC-1/PD-1: For ether sent with no calldata, which the treasury accepts and keeps
- [PASS] test_check_stranger_changes_nothing_renounceOwnership(address,uint256) (paths: 3, time: 0.05s) -- AC-1/PD-1: For renounceOwnership
- [PASS] test_check_stranger_changes_nothing_setBeneficiary(address,uint256,address) (paths: 4, time: 0.06s) -- AC-1/PD-1: For setBeneficiary

- [PASS] test_check_stranger_changes_nothing_sweep(address,uint256) (paths: 3, time: 0.06s) -- AC-1/PD-1: For sweep, the one call here that lands
- [PASS] test_check_stranger_changes_nothing_sweepToken(address,uint256,address) (paths: 6, time: 0.09s) -- AC-1/PD-1: For sweepToken, pointed at any plain address or at one of the system's own contracts
- [PASS] test_check_stranger_changes_nothing_transferOwnership(address,uint256,address) (paths: 4, time: 0.06s) -- AC-1/PD-1: For transferOwnership

GlamToonHistorySymbolicTest

File: test/dinosat/GlamToonHistorySymbolic.t.sol

11 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_history_survives_anySelector(address,bytes4) (paths: 68, time: 1.59s) -- AC-3/TOK-1: For any four byte selector sent alone
- [PASS] test_check_history_survives_approve(address,address,uint256) (paths: 7, time: 0.14s) -- AC-3/TOK-1: For approve
- [PASS] test_check_history_survives_mintBlue(address,uint256,uint256) (paths: 5, time: 0.15s) -- AC-3/TOK-1: For mintBlue
- [PASS] test_check_history_survives_mintBlueFor(address,address,uint256) (paths: 8, time: 0.29s) -- AC-3/TOK-1: For mintBlueFor
- [PASS] test_check_history_survives_mintGreen(address,uint256) (paths: 5, time: 0.16s) -- AC-3/TOK-1: For mintGreen
- [PASS] test_check_history_survives_plainEther(address) (paths: 4, time: 0.08s) -- AC-3/TOK-1: For ether sent with no calldata
- [PASS] test_check_history_survives_safeTransferFrom(address,address,address,uint256) (paths: 8, time: 0.24s) -- AC-3/TOK-1: For safeTransferFrom without data
- [PASS] test_check_history_survives_safeTransferFromWithData(address,address,address,uint256) (paths: 9, time: 0.30s) -- AC-3/TOK-1: For safeTransferFrom with data
- [PASS] test_check_history_survives_setApprovalForAll(address,address,bool) (paths: 5, time: 0.08s) -- AC-3/TOK-1: For setApprovalForAll
- [PASS] test_check_history_survives_transferFrom(address,address,address,uint256) (paths: 8, time: 0.21s) -- AC-3/TOK-1: For transferFrom, from any holder to anywhere
- [PASS] test_check_history_survives_withdraw(address) (paths: 4, time: 0.09s) -- AC-3/TOK-1: For withdraw

GlamToonEtherFlowSymbolicTest

File: test/dinosat/GlamToonEtherFlowSymbolic.t.sol

11 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_ether_stays_put_anySelector(address,uint256,uint256,bytes4) (paths: 133, time: 1.12s) -- PD-1/CC-3: For any four byte selector sent alone: every function that takes no argument, view or not, and every selector the contract does not have
- [PASS] test_check_ether_stays_put_approve(address,uint256,uint256,address,uint256) (paths: 7, time: 0.08s) -- PD-1/CC-3: For approve
- [PASS] test_check_ether_stays_put_mintBlue(address,uint256,uint256,uint256,uint256) (paths: 10, time: 0.14s) -- PD-1/CC-3: For mintBlue
- [PASS] test_check_ether_stays_put_mintBlueFor(address,uint256,uint256,address,uint256) (paths: 15, time: 0.21s) -- PD-1/CC-3: For mintBlueFor

- [PASS] test_check_ether_stays_put_mintGreen(address,uint256,uint256,uint256) (paths: 7, time: 0.12s) -- PD-1/CC-3: For mintGreen
- [PASS] test_check_ether_stays_put_plainEther(address,uint256,uint256) (paths: 3, time: 0.04s) -- PD-1/CC-3: For ether sent with no calldata
- [PASS] test_check_ether_stays_put_safeTransferFrom(address,uint256,uint256,address,uint256) (paths: 10, time: 0.16s) -- PD-1/CC-3: For safeTransferFrom without data
- [PASS] test_check_ether_stays_put_safeTransferFromWithData(address,uint256,uint256,address,uint256) (paths: 10, time: 0.16s) -- PD-1/CC-3: For safeTransferFrom with data
- [PASS] test_check_ether_stays_put_setApprovalForAll(address,uint256,uint256,address,bool) (paths: 7, time: 0.07s) -- PD-1/CC-3: For setApprovalForAll
- [PASS] test_check_ether_stays_put_transferFrom(address,uint256,uint256,address,uint256) (paths: 8, time: 0.12s) -- PD-1/CC-3: For transferFrom
- [PASS] test_check_ether_stays_put_withdraw(address,uint256,uint256) (paths: 5, time: 0.06s) -- PD-1/CC-3: For withdraw

TraitDataSymbolicTest

File: test/dinosat/TraitDataSymbolic.t.sol

8 formally proven, 2 fuzz-verified

Proven:

- [PASS] test_check_chunk0_refuses_characters_outside(uint256) (paths: 1, time: 0.00s) -- SM-4: The first chunk refuses every character it does not carry with OutOfChunk, rather than reading past its data
- [PASS] test_check_chunk1_refuses_characters_outside(uint256) (paths: 2, time: 0.01s) -- SM-4: The second chunk refuses every character below 5000 and from 10000 up
- [PASS] test_check_data_matches_the_schema() (paths: 1, time: 0.12s) -- NFT-6: Each data contract hashes to what TraitSchema expects, holds as many bytes as its layout says, and the name table has the slot count the lemmas assume
- [PASS] test_check_names_refuse_a_slot_outside(uint256) (paths: 1, time: 0.00s) -- SM-4: The name table refuses a slot it does not have with OutOfRange
- [PASS] test_check_token_reads_the_right_record_0() (paths: 1, time: 0.02s) -- NFT-6: The token reads the first record of the first chunk, and it is the bytes at that position
- [PASS] test_check_token_reads_the_right_record_4999() (paths: 1, time: 0.07s) -- NFT-6: The last record of the first chunk
- [PASS] test_check_token_reads_the_right_record_5000() (paths: 1, time: 0.01s) -- NFT-6: The first record of the second chunk
- [PASS] test_check_token_reads_the_right_record_9999() (paths: 1, time: 0.01s) -- NFT-6: The last record of the second chunk

Fuzz-verified:

- [FUZZ] test_check_token_reads_the_right_record(uint256) (1,000,000 runs, 0 failures) -- NFT-6: For every character, the record the token reports is the two bytes at that character's position in the right chunk. Halmos: ERROR (symbolic memory offset).
- [FUZZ] test_check_traits_are_the_fields_of_the_record(uint256) (1,000,000 runs, 0 failures) -- NFT-6: For every character, the four trait values are the four fields of its record, each at most fifteen. Halmos: ERROR (symbolic memory offset).

GlamToonMetadataSymbolicTest

File: test/dinosat/GlamToonMetadataSymbolic.t.sol

9 formally proven, 3 fuzz-verified

Proven:

- [PASS] test_check_art_addresses_refuse_an_unknown_character(uint256,bool) (paths: 2, time: 0.01s) -- NFT-5: The four art address functions refuse a character past the collection with InvalidImageId, in either edition
- [PASS] test_check_collection_metadata_is_fixed() (paths: 1, time: 0.07s) -- NFT-5: The art identifiers the contract reports are the ones in its source, and the collection metadata is its JSON in base64
- [PASS] test_check_tokenURI_refuses_an_unminted_token(uint256) (paths: 1, time: 0.00s) -- NFT-5: tokenURI refuses a token that was never minted, for every id, as ERC-721 requires
- [PASS] test_check_token_json_answers_0() (paths: 1, time: 0.05s) -- NFT-5: The first blue has well formed metadata
- [PASS] test_check_token_json_answers_10000() (paths: 1, time: 0.03s) -- NFT-5: The first green of character 0
- [PASS] test_check_token_json_answers_19999() (paths: 1, time: 0.02s) -- NFT-5: The first green of the last character
- [PASS] test_check_token_json_answers_90000() (paths: 1, time: 0.02s) -- NFT-5: Generation nine, where the Generation trait starts to read "9+"
- [PASS] test_check_token_json_answers_9999() (paths: 1, time: 0.02s) -- NFT-5: The last blue
- [PASS] test_check_token_json_answers_max() (paths: 1, time: 0.03s) -- NFT-5: The largest token id: metadata still answers

Fuzz-verified:

- [FUZZ] test_check_art_addresses_have_the_right_shape(uint256,bool) (1,000,000 runs, 0 failures) -- NFT-5: For every character and either edition, each of the four art addresses is the scheme, the root, the edition's folder, the character's number in decimal and the extension, nothing else. Halmos: ERROR (symbolic copy size).
- [FUZZ] test_check_tokenURI_wraps_the_json(uint256) (1,000,000 runs, 0 failures) -- NFT-5: For a minted blue and its first green, tokenURI is the token's JSON in base64 behind the data URI prefix. Halmos: no result, stopped after 120 s of path exploration.
- [FUZZ] test_check_token_json_answers_for_every_id(uint256) (1,000,000 runs, 0 failures) -- NFT-5: tokenJSON answers for every token id there is, minted or not, and what it answers opens and closes as one JSON object. Halmos: no result, stopped after 120 s of path exploration.

GlamToonPackingSymbolicTest

File: test/dinosat/GlamToonPackingSymbolic.t.sol

6 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_a_blue_id_is_the_character_number(uint256) (paths: 3, time: 0.36s) -- NFT-1: A blue's token id is its character's number, and decodes back to that character at generation zero
- [PASS] test_check_decode_always_names_a_real_character(uint256) (paths: 2, time: 0.87s) -- NFT-2: Decoding any token id at all names a character inside the collection, and the two single readers agree with decode
- [PASS] test_check_decode_inverts_tokenIdFor(uint256,uint256) (paths: 3, time: 8.68s) -- NFT-2: Decode, imageIdOf and generationOf give back the pair tokenIdFor was given
- [PASS] test_check_isBlue_is_the_first_ten_thousand_ids(uint256) (paths: 1, time: 0.00s) -- NFT-3: isBlue is true for the first ten thousand ids and for no other
- [PASS] test_check_tokenIdFor_answers_the_formula(uint256,uint256) (paths: 3, time: 11.25s) -- NFT-1/AS-1: tokenIdFor answers, rather than reverts, for every character and generation in range, and what it answers is the generation times ten thousand plus the character, unwrapped
- [PASS] test_check_tokenIdFor_refuses_a_generation_that_would_overflow(uint256,uint256) (paths: 4, time: 6.50s) -- AS-1: A generation so large that its id would not fit a word is refused

GlamToonTraitReadersSymbolicTest

File: test/dinosat/GlamToonTraitReadersSymbolic.t.sol

9 formally proven, 0 fuzz-verified

Proven:

- [PASS] test_check_all_traits_is_both_chunks_in_order() (paths: 1, time: 0.39s) -- NFT-6: allTraits is the first chunk followed by the second, two bytes for each of the ten thousand characters
- [PASS] test_check_category_names_are_the_first_slots() (paths: 1, time: 0.02s) -- NFT-6: categoryNames is the first four slots of the name table, in record order, and none of them is empty
- [PASS] test_check_trait_chunk_is_the_data_contract_0() (paths: 1, time: 0.15s) -- NFT-6: traitChunk answers the first data contract's records, whole
- [PASS] test_check_trait_chunk_is_the_data_contract_1() (paths: 1, time: 0.15s) -- NFT-6: traitChunk answers the second data contract's records, whole
- [PASS] test_check_trait_names_is_the_name_table() (paths: 1, time: 0.01s) -- NFT-6: traitNames is the name table the constructor pinned
- [PASS] test_check_value_names_are_the_category_table_0() (paths: 1, time: 0.12s) -- NFT-6: valueNames answers the sixteen slots of the category's value table, in order
- [PASS] test_check_value_names_are_the_category_table_1() (paths: 1, time: 0.06s) -- NFT-6: The same
- [PASS] test_check_value_names_are_the_category_table_2() (paths: 1, time: 0.06s) -- NFT-6: The same
- [PASS] test_check_value_names_are_the_category_table_3() (paths: 1, time: 0.06s) -- NFT-6: The same

Properties Verified

The number after each status is how many tests carry that property. IDs are from the DinoSAT property catalog; NFT-1 to NFT-6 are specific to this protocol and defined by the rows below.

P0 -- Critical

ID	Property	Function(s)	Status	Technique
AC-1	Unauthorised caller is refused	setBeneficiary, transferOwnership, transferFrom, approve	PROVEN (17)	DIRECT, ENUMERATE
AC-2	Open functions grant no privilege	withdraw, sweep, owner, mintBlue	PROVEN (7)	DIRECT
AC-3	What is fixed or recorded cannot be undone	constructor, renounceOwnership, all writers	PROVEN (17)	DIRECT, ENUMERATE
AC-4	Pending owner clears on handover	acceptOwnership, transferOwnership	PROVEN (2)	DIRECT
AC-5	Ownership moves only in two steps	transferOwnership, acceptOwnership	PROVEN (3)	DIRECT
AS-1	No overflow in price, token id or royalty	priceAt, tokenIdFor, royaltyInfo	PROVEN (6)	DIRECT
AS-3	No subtraction underflow	_mintBlue, blueTakenBits, record	PROVEN (4)	DIRECT, ENUMERATE
AS-4	Casts cannot truncate	owner, traitsOf	PROVEN (7)	DIRECT

P1 -- High

ID	Property	Function(s)	Status	Technique
FB-1	Royalty never exceeds the sale price	royaltyInfo	PROVEN (1)	DIRECT
FB-2	Royalty rate is below 100%	royaltyInfo	PROVEN (1)	DIRECT
FB-3	Nobody pays less than the tier price	priceAt, mintBlue, mintBlueFor	PROVEN (11)	DIRECT, DECOMPOSE
FB-4	Price never above 0.04 ether or the ceiling	priceAt, mintBlue, mintBlueFor	PROVEN (9)	DIRECT, DECOMPOSE
FB-5	Price and royalty are monotone	priceAt, royaltyInfo	PROVEN (1) + FUZZ (1)	DIRECT, FUZZ
TOK-1	Supply counters add up	totalSupply, mintBlue, mintGreen	PROVEN (12)	DIRECT, ENUMERATE
TOK-2	A transfer conserves balances and supply	transferFrom	PROVEN (2)	DIRECT
TOK-3	Blue supply is capped at 10,000	mintBlue, mintBlueFor, priceAt	PROVEN (5)	DIRECT
TOK-4	No token can be burned	transferFrom, mintBlueFor	PROVEN (2)	DIRECT
TOK-5	An approval is not a transfer	approve, transferFrom	PROVEN (2)	DIRECT
NFT-1	No two tokens share an id	tokenIdFor	PROVEN (5)	DIRECT, DECOMPOSE
NFT-2	Decoding undoes encoding	decode, imageIdOf, generationOf	PROVEN (5)	DIRECT
NFT-3	A green never lands on a blue id	tokenIdFor, isBlue, mintGreen	PROVEN (4)	DIRECT
NFT-4	Re-entry through hook or refund is refused	mintBlue, mintGreen, withdraw	PROVEN (4)	DIRECT

P2 -- Medium

ID	Property	Function(s)	Status	Technique
PD-1	Ether is conserved: price + change = sent	mintBlue, withdraw, sweep, all writers	PROVEN (34)	DIRECT, ENUMERATE
PD-2	Each part is bounded by the total	royaltyInfo	PROVEN (1)	DIRECT
PD-3	Royalty is exactly 2%, rounded down	royaltyInfo	PROVEN (2) + FUZZ (2)	DIRECT, FUZZ
SM-1	Gating: sale start, one blue, one green each	mintBlue, mintGreen, constructor	PROVEN (13)	DIRECT
SM-4	Readers refuse inputs out of range	blueTakenBits, traitChunk, record, ...	PROVEN (20)	DIRECT, ENUMERATE
SM-5	Generations and price tiers advance in order	mintGreen, currentPrice	PROVEN (5)	DIRECT
NFT-5	Metadata answers and names the right art	tokenJSON, tokenURI, imageURI, ...	PROVEN (10) + FUZZ (3)	DIRECT, ENUMERATE, FUZZ
NFT-6	Trait data is pinned and read correctly	constructor, recordOf, traitsOf, ...	PROVEN (21) + FUZZ (2)	DIRECT, ENUMERATE, FUZZ

Cross-Contract

ID	Property	Contracts	Status	Technique
CC-1	Bitmap, owner and counters agree	GlamToon	PROVEN (6)	DIRECT
CC-2	owner() label follows the treasury's owner	GlamToon, Treasury	PROVEN (2)	DIRECT
CC-3	Proceeds flow token -> treasury -> beneficiary	GlamToon, Treasury	PROVEN (25)	DIRECT, ENUMERATE

Coverage Gap Analysis

Counted by following each proof through its helpers and collecting every contract function it names. The any-selector proofs are not counted for anything.

GlamToon: the sale, the money and the tokens

Function	Tests	Gap
constructor	10	--
mintBlue	37	--
mintBlueFor	7	--
mintGreen	15	--
withdraw	13	--
owner	29	--
currentPrice	2	--
priceAt	12	--
supportsInterface	1	--
totalSupply	16	--
royaltyInfo	24	--
approve	6	--
setApprovalForAll	3	--
transferFrom	13	--
safeTransferFrom	6	--
balanceOf	10	--
ownerOf	39	--
getApproved	3	--
isApprovedForAll	1	--
blueMinted	42	--
greenMinted	26	--
greenCount	20	--
greenTaken	21	--

GlamToon: the readers

Function	Tests	Gap
tokenIdFor	12	--
decode	10	--
imageIdOf	2	--
generationOf	2	--
isBlue	4	--
tokenURI	2	--
tokenJSON	8	--
contractURI	1	--
collectionJSON	1	--
imageURI	3	--
optimizedURI	2	--
arweaveURI	2	--
arweaveOptimizedURI	2	--
ipfsRoot	1	--
arweaveManifest	1	--
tokenBase	1	--
traitsOf	2	--
traitsBatch	1	--
recordOf	7	--
traitChunk	3	--
allTraits	1	--
traitNames	1	--
categoryNames	1	--
valueNames	5	--
blueTaken	20	--
blueTakenBits	26	--
blueBitmapWords	1	--
greenTakenBatch	1	--
greenCountBatch	1	--

Treasury

Function	Tests	Gap
constructor	1	--
receive	24	--
renounceOwnership	2	--
setBeneficiary	8	--
sweep	11	--
sweepToken	7	--
transferOwnership	7	--
acceptOwnership	5	--
pendingOwner	13	--
beneficiary	14	--

The trait data contracts and the schema library

Function	Tests	Gap
TraitData.record	7	--
TraitData.data	9	--
TraitData.dataHash	1	--
TraitNames.namesAt	6	--
TraitNames.namesHash	1	--
TraitSchema.shifts	5	--
TraitSchema.masks	9	--
TraitSchema.valueBase	6	--

Not covered by a proof

What	Tests	Gap	Reason
GlamToon and Treasury events	0	P2	Halmos cannot observe events
constructor: payee == token itself	0	P2	Needs a predicted CREATE address
batch readers, length > 1	0	P2	Symbolic array length; length 1 proved
_isNone, _generationLabel, _uri, _checked	--	--	Private, reached through the readers
script/Deploy.s.sol	0	--	Out of scope: DinoSAT verifies src/

> All P0 gaps are zero. The three P2 gaps are things Halmos cannot express. Each is covered by the project's own unit tests (ConstructorPins.t.sol, Events.t.sol and the batch reader tests), which this run did not modify.

Methodology

Symbolic Execution

Halmos translates EVM bytecode into SMT (Satisfiability Modulo Theories) constraints. For each assert() statement, Z3 searches for any input assignment that violates the assertion. If no such assignment exists, the property is mathematically proven for ALL inputs satisfying the vm.assume() preconditions.

This run used Halmos 0.3.3 with --solver z3 (Z3 4.12.6), because Halmos 0.3 no longer defaults to Z3. The proofs execute the real contracts: each stateful test contract deploys both trait chunks, the name table, a Treasury and a GlamToon wired as script/Deploy.s.sol wires them, then calls the deployed bytecode. Every call a proof is about is made low level and its success or failure asserted, so a function that refused everything could not pass by having its paths dropped.

Commands, from contracts/, once per test contract and one contract at a time:

```
FOUNDRY_PROFILE=dinosat halmos --contract <Name>Test --function "test_check_" \
  --forge-build-out out-dinosat --solver z3 --solver-timeout-assertion 30000
```

4 proofs timed out at 30 seconds and were retried once with --solver-timeout-assertion 60000, the one extended attempt DinoSAT allows before a property is left to fuzzing. 1 of them passed (test_check_royalty_always_answers_in_range, 36 s); the other 3 timed out again.

2 proofs build strings from a symbolic number, which makes Halmos explore paths without end and without ever reaching the solver, so no solver timeout applies. Each was run by itself and stopped after 120 seconds of wall clock with no result. They were classified for fuzzing before the run and are fuzz-verified.

--loop 5 was added for the five contracts whose tests contain loops. The profile is [profile.dinosat] in foundry.toml: optimizer off, evm_version = cancun (the reentrancy guard uses transient storage, which shanghai lacks), ast = true.

Proof Techniques Applied

Technique	Count	Applied to
Cross-multiplication	0	--
Boundary parameterization	0	--
Concrete enumeration	51	AC-1, AC-3, AS-3, CC-3, NFT-5, NFT-6, PD-1, SM-4, TOK-1
Inline mulDiv	0	--
Lemma decomposition	5	FB-3, FB-4, NFT-1
Monotonicity	3	FB-5, NFT-1
Floor division identity	9	CC-1, NFT-2, PD-2, PD-3
Abstract specification	31	AC-3, AS-1, AS-3, CC-1, FB-1, FB-2, FB-3, FB-4, FB-5, and 8 more
Guard, outcome asserted (T3)	52	AC-1, AC-2, AC-3, AC-5, AS-4, CC-3, FB-3, FB-4, NFT-4, and 7 more
State gating (T12)	13	AS-3, FB-3, FB-4, SM-1, SM-4, SM-5, TOK-3
Conservation (T14)	34	AC-2, AS-3, CC-3, PD-1, TOK-2
Stateful interaction (T16)	30	AC-2, AC-3, AC-4, AC-5, AS-1, CC-1, CC-2, CC-3, FB-3, and 8 more
Constant bound (T2)	36	AC-3, AS-1, AS-4, CC-1, CC-2, FB-1, FB-2, FB-3, FB-4, and 5 more

The first eight rows are DinoSAT's proof techniques; the last five are its test templates, which is how the remaining proofs are built. A test can appear in more than one row. No property needed cross-multiplication, boundary parameterization or an inline mulDiv: the contracts contain no Math.mulDiv, no Math.sqrt and no ratio comparison.

Z3-Hostile Patterns Detected

src/ itself contains no assembly, no mulDiv, no sqrt, no symbolic exponent and no else branch. The hostile patterns are in what it calls and in two constant divisors.

Pattern	Location	Strategy	Result
Strings.toString, Base64 assembly loops	GlamToon.sol:387-500	ENUMERATE ids + FUZZ	PROVEN (6) + FUZZ (3)
Symbolic index into 10,000 constant bytes	TraitData0.sol:55-60	ENUMERATE ends + FUZZ	PROVEN (4) + FUZZ (2)
/ and % by 10,000, wide numerator	GlamToon.sol:351-352	DIRECT, generation < 2^16	PROVEN (3)
Checked generation * 10000	GlamToon.sol:341	DECOMPOSE, division-free	PROVEN (5)
salePrice * 200 / 10000	ERC2981.sol via royaltyInfo	formula match + FUZZ	PROVEN (9) + FUZZ (3)
Loop over a symbolic word count	GlamToon.sol:642-648	DIRECT guard + ENUMERATE	PROVEN (4)

Fuzz Testing (Complementary)

Properties are fuzz-verified via two Forge passes, both on the default profile: Solidity 0.8.37, prague, optimizer on with 200 runs, via-IR. That is the build that was deployed, where the Halmos proofs run an unoptimized cancun build, so the two passes cover each other's blind side.

- Pass 1 -- Fuzz wrappers (test_fuzz_): 1,000,000 runs per test. Uses bound() for near-zero rejection and real input coverage. 119 wrappers, 119 passed.
- Pass 2 -- Halmos tests (test_check_): 100,000 runs per test. High rejection expected due to tight vm.assume bounds -- primary coverage comes from Pass 1. 222 tests: 104 passed, 118 stopped on rejected inputs, 0 failed for another reason.
- Failures: 0

```
forge test --match-path 'test/dinosat/*.t.sol' \
  --no-match-path 'test/none/*' --match-test test_fuzz_ \
  --fuzz-runs 1000000 --fuzz-seed 0x44696e6f534154
forge test --match-path 'test/dinosat/*.t.sol' \
  --no-match-path 'test/none/*' --match-test test_check_ \
  --fuzz-runs 100000 --fuzz-seed 0x44696e6f534154
```

foundry.toml excludes test/dinosat from a plain forge test, so the project's own suite is unchanged: run after this work, it reported 40 suites and 695 tests passed, 0 failed. --no-match-path overrides the exclusion for these two commands. A test with no vm.assume numeric bound has no wrapper and is fuzzed by Pass 2 directly.

Vacuity Check

A proof whose assumptions contradict each other, or whose every path reverts, passes while proving nothing. To rule that out, 217 of the 222 proofs were copied with assert(false) appended as the last statement and run through Halmos again. A vacuous proof has no path that reaches that statement, so its copy would pass. 217 of the 217 copies failed with a counterexample, which shows the proof each mirrors has at least one complete path.

The other 5 proofs have no copy: they are the ones Halmos cannot execute symbolically at all (a symbolic memory offset, a symbolic copy size, or unbounded path exploration), so a copy would stop in the same place. They are fuzz-verified. The copies lived outside test/ and were deleted after the run.

Seeded-Defect Check

As a check on the proofs rather than on the contracts, 17 defects were seeded one at a time into a scratch copy of src/ and the test contract that should notice was run against each. The real src/ was only read. 17 of 17 were caught: 16 by at least one failing proof, and 1 because the token's own constructor refused to deploy with the defect in place.

Seeded defect	Run against	Proofs that failed
The buyer's change is never sent back	GlamToonBlueSaleSymbolicTest	1
The price tier is 2501 sales wide instead of 2500	GlamToonPriceSymbolicTest	3
A green mint does not record the claim	GlamToonGreenSymbolicTest	3
The already-sold check is removed from the blue sale	GlamToonBlueGuardsSymbolicTest	2
The supply cap is 10,001 instead of 10,000	GlamToonBlueGuardsSymbolicTest	1
withdraw pays its caller instead of the treasury	GlamToonWithdrawSymbolicTest	3
mintGreen loses its reentrancy lock	GlamToonReentrancySymbolicTest	2
Token ids are packed by 9,999 instead of 10,000	GlamToonPackingSymbolicTest	2
owner() truncates a dirty word into an address	GlamToonOwnerDecodingSymbolicTest	1
The bitmap range check is off by one	GlamToonViewGuardsSymbolicTest	2
The sale opens one second early	GlamToonBlueGuardsSymbolicTest	3
Anyone can repoint the treasury	TreasuryAccessSymbolicTest	3
sweep pays its caller instead of the beneficiary	TreasurySweepSymbolicTest	2
The treasury can be renounced	TreasuryAccessSymbolicTest	1
The royalty is 20% instead of 2%	GlamToonRoyaltySymbolicTest	2
The second trait chunk is read for every character	TraitDataSymbolicTest	all: constructor refuses to deploy
A trait record's two bytes are read in the wrong order	TraitDataSymbolicTest	1

The first run of this check left one defect uncaught, the supply cap of 10,001. priceAt checks the cap again, so the sale was still refused, but once the character was taken the reason given changed from SoldOut to BlueAlreadyTaken, and no proof pinned the order of the reasons. test_check_refusal_reason_follows_the_guard_order was added to pin all six in order, proved, and the check run again; the table is that second run.

Input Domain Bounds

Variable	Range	Justification
imageId	[0, 9999]	IMAGE_COUNT, checked at GlamToon.sol:243, 272, 340
sold (blueMinted)	[0, 9999], or >= 10000	BLUE_SUPPLY, checked at GlamToon.sol:246, 328
ether sent or held	[0, 2^128 - 1] wei	More ether than exists; keeps sums in a word
maxPrice	any uint256	No bound
block.timestamp	[0, 2^64 - 1]	SALE_START is a uint64
generation, no division	[0, 2^200)	A count of green mints; unreachable
generation, with division	[0, 2^16)	Solver limit; wider range fuzzed
tokenId for recompose	[0, 2^32)	Solver limit; every id fuzzed
royalty salePrice	[0, 1e27] wei	A billion ether, more than exists
ERC-20 amounts	[1, 2^128 - 1]	Keeps balance sums in a word
caller, as an account	no code, > 0xffff	Not a precompile, not the test harness
caller, in guard proofs	any but the zero address	The zero address has no key
token id, word, selector, hash	any value	No bound

Soundness Caveats

Proof Domain Boundaries

Formal proofs are valid ONLY within the vm.assume() input domains documented in the Input Domain Bounds section. Values outside these ranges are not covered by any proof. Two bounds are solver limits, not protocol limits: the decode proofs stop at generation 65,535 and the recompose lemma at token id 2^32. Above them the property is fuzz-verified, and the property that matters for minting, that ids never collide, is proved without division up to generation 2^200.

Abstract Specification Equivalence

The five lemma contracts restate contract arithmetic instead of calling it. No Math.mulDiv is involved, so no inline-arithmetic equivalence is assumed. Each restated expression is tied to the deployed code by a stateful proof: GlamToonPriceSymbolicTest for the price and its constants, GlamToonPackingSymbolicTest for token ids, GlamToonRoyaltySymbolicTest for the royalty, and GlamToonBlueSaleSymbolicTest for the bitmap. TraitSchemaLemmasTest calls the TraitSchema library itself.

Compositional Soundness

Decomposed proofs establish per-branch correctness. The composition requires that branch conditions partition the input space exhaustively. Each decomposed property documents its branch partition and composition status in Detailed Results. The any-call proofs enumerate the state-changing functions by hand; test/unit pins the compiled ABI to a written

list, so a function missing from the enumeration would have to be missing from that list too.

Halmos Verification Scope

Halmos verifies single-transaction properties. It does NOT verify:

- Reentrancy: callback-based attacks in general. Four concrete attacker contracts are driven symbolically here; arbitrary attacker code is not.
- Gas limits: all paths explored regardless of gas cost.
- Storage layout: blueMinted is set with vm.store at slot 8 and read back in every proof that uses it, so a moved layout fails rather than passes.
- Long sequences: proofs make one to four calls. Invariants over long call sequences are the project's invariant campaigns' job.
- Events: Halmos cannot observe emitted events.
- ERC-20 transfer quirks: only a standard token, a no-return token and a false-returning token are covered; fee-on-transfer and rebasing tokens are not.

The proofs run a cancun, optimizer-off build. That build of GlamToon exceeds the 24,576 byte limit and could not itself be deployed; the deployed contract is the prague, optimizer-on, via-IR build, which the two fuzz passes exercise. A compiler bug specific to the optimized build is outside what the proofs cover.

Fuzz Limitations

FUZZ-classified properties provide empirical evidence from 1,000,000 random inputs. A counterexample may exist outside the sampled space. Where a FUZZ property is also covered by decomposed lemma proofs, this is documented as "PROVEN via composition" in Detailed Results.

Conclusion

Total properties:	222
Formally proven (Halmos/Z3):	214 -- mathematical proof within the stated domains
Proven via composition:	2 -- the price schedule over its four tiers; id uniqueness to generation 2^200
Fuzz-verified (Forge):	8 -- 1,000,000 runs, 0 failures
Counterexamples found:	0

No vulnerabilities were discovered during formal verification.

The contracts are deployed and immutable, so src/ was not modified by this run. The only changes are the files under test/dinosat/, one profile and one exclusion line in foundry.toml, and ignore entries for that profile's two build folders.

Disclaimer

This report was generated by DinoSAT, an automated formal verification tool. It is provided "as is" without warranty of any kind, express or implied. This report does not constitute a security audit, and does not guarantee the absence of vulnerabilities, bugs, or exploits in the verified contracts.

Formal proofs are valid only within the documented input domains (vm.assume bounds). Fuzz-verified properties are empirical observations, not mathematical proofs. Properties outside the scope of this report are not covered. These include: reentrancy (Halmos models single-call execution, not callback chains), gas optimization (all paths explored regardless of gas cost), business logic correctness (requires domain-specific invariants beyond arithmetic), economic incentive alignment (game-theoretic analysis is outside SMT solving), and cross-contract interactions (Halmos verifies contracts in isolation -- multi-contract state dependencies require stateful integration testing).

This tool is free to use. Users assume all risk associated with reliance on this report. The authors accept no liability for any loss or damage arising from the use of this software or the information contained in this report.

https://dinosat.com